

MOUNTING THE DRIVE

```
import numpy as np
import pandas as pd
import os
for dirname, __, filename in os.walk('/kaggle/input') :
    for filename in filename :
        print(os.path.join(dirname, filename))
```

IMPORTING LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

READING DATA FRAME

```
path='/kaggle/input/car-data/CarPrice_Assignment.csv'
df=pd.read_csv(path)
```

```
df.head()
```

DATA PREPROCESSING

```
df.info()
```

```
df['highwaympg']
```

```
df.isnull().sum()
```

```
df.describe()
```

```
from ast import Return
numerical_columns=['wheelbase','carlength','carwidth','carheight','curbweight','enginesize','horsepower','highwaympg','price']
```

```
def remove_outliers(df, columns):
    for column in columns:
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1
numerical_columns=['wheelbase','carlength','carwidth','carheight','curbweight','enginesize','horsepower','highwaympg','price']
```

```
def remove_outliers(df, columns):
    for column in columns:
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[~((df[column]<lower_bound)|(df[column]>upper_bound)).any(axis=1)]
    return df
df=remove_outliers(df,numerical_columns)
df_copy=df
```

TRAIN THE MODEL

a) Linear Regression

```
linReg=LinearRegression()  
linReg.fit(x_train,y_train)
```

```
y_pred=linReg.predict(x_test)  
y_pred
```

EVALUATE THE MODEL

```
linReg.score(x_train,y_train)  
linReg.score(x_test,y_test)
```

```
sns.scatterplot(x=y_test,y=y_pred,color='blue',label='Actual Data points')  
plt.plot([min(y_test),max(y_test),max(y_test)],color='red',label='IdealLine')  
plt.legend()  
plt.show()
```

```
predictions=linReg.predict(x_test)  
actual_values=y_test  
plt.scatter(predictions,actual_values,alpha=.7,color='b')  
plt.xlabel('Predicted Price')  
plt.ylabel('Actual Price')  
plt.title('Actual Prices vs Predicted Prices')  
plt.show()
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error  
mse=mean_absolute_error(y_test,y_pred)  
mpe=abs(np.mean((y_test-y_pred)/y_test))*100 print(mpe)
```

PREDICTIONS

a) Dummy Variables

```
def dummies(x,df):  
    temp=pd.get_dummies(df[x],drop_first=True)  
    df=pd.concat([df,temp],axis=1)  
    df.drop([x],axis=1,inplace=True)  
    return df
```

```
cars_lr=dummies('fueltype',cars_lr)  
cars_lr=dummies('aspiration',cars_lr)  
cars_lr=dummies('carbody',cars_lr)  
cars_lr=dummies('drivewheel',cars_lr)  
cars_lr=dummies('enginetype',cars_lr)  
cars_lr=dummies('carsrange',cars_lr)
```

```
cars_lr.head()
```

```
cars_lr.shape()
```

TRAINING AND TESTING DATA

a)Splitting data

```
from sklearn.model_selection import train_test_split  
np.random.seed(0)  
df_train,df_test=train_test_split(cars_lr,train_size=0.7,test_size=0.3,random_state=100)
```

b) Feature Scaling

```
from sklearn.preprocessing import MinMaxScaler
scaler =MinMaxScaler()
num_vars=['wheelbase','curbweight','enginesize','boreratio','horsepower','fueleconomy','carlength','carweight','price']
df_train[num_vars]=scaler.fit_transform(df_train[num_vars])

df_train.head()

df_train.describe()
```